Common Table Expressions (CTEs).

PAGE NO.....
DATE / /

→ A CTE is a temporary result set created using **with** keyword. It exists only during query execution and improves readability by breaking a query into a smaller, reusable parts.

When to Use CTEs?

- When dealing with recursive queries (eg. hierarchical data like organizational structures).
- When you need to use the same subquery multiple times.
- To improve query readability by replacing complex subqueries.

(employees → table_name)

employee_id	name	department_id	manager_id	salary
1	Alice	1	Null	7000.00
2	Bob	1	1	5000.00
3	Charlie	1	1	4000.00
4	David	2	Null	8000.00
5	Eva	2	4	5500.00
6	Frank	2	4	4500.00
7	Grace	3	Null	6000.00

int (primary key) | varchar(100) | int | int | Decimal(10,2)

dhawaldongaonkar

CTEs table is not updatable

You can't perform Update, Delete, or Insert.

```
with department_avg As (
    Select department_id, AVG(salary) As avg_salary
    From employees
    Group by department_id
)

Select e.employee_id, e.name, e.salary, e.department_id
From employee e
Join department_avg d
ON e.department_id = d.department_id
Where e.salary > d.avg_salary;
```

Output

employee_id	name	salary	department_id
1	Alice	7000.00	1
4	David	8000.00	2

Why does [department_avg] behaves like table?

1. CTEs ([with] statements) generates a temporary result set:

   → department_avg contains the department_id and the average salary per department.

   → Think of it as a "virtual table" that exists only for the durations of the query.

2. The [Join] works because [department_avg] acts like a table.

   → employees (base table) has employee_id, name, salary, department_id.

PAGE NO.....
DATE / /

- department_avg (CTE) has department_id, avg_salary
- Since both have department_id, they can be joined like two tables.

Difference between a Table and a User-Defined Function

Feature	CTE (department_avg)	User-Defined function
Behavior	Temporary result set (like a table)	Return a scalar value, table, or row
Scope	Exist only within the query	can be reused across multiple queries.
use Case	Complex queries with grouping, ranking etc	Encapsulating logic for reusability.
Joinable?	Yes, acts as a table	Only if it's a table-valued function.

How it works?

1. The CTE (department_avg) calculates the average salary for the each department.
2. The main query compares employee salaries with the department's average.

PAGE NO.....
DATE / /

Temporary Tables.

→ A Temporary Table is a table that stores data temporarily and exists only for the duration of the session or transaction. Unlike CTEs, temporary tables persist until manually dropped.

When to Use Temporary Table?

→ When dealing with large datasets that need to be used multiple times.

→ To store intermediate results for complex queries.

→ When you need to modify the dataset before final querying.

Example 1:

```
Create Temporary Table high_salary_employees As
Select * From employee Where salary > 5000;

Select * From high_salary_employees;
```

- How it works

→ Creates a temporary table (high_salary_employees) storing employees earning above 5000.

→ The data is used for further analysis without modifying the original employee table.

dhawaldongaonkar

PAGE NO.....
DATE / /

Example2 : Using Temporary Tables for Performance Optimization.

```
Create Temporary Table department_salaries As
Select department_id, SUM(salary) As total_salary
From employees.
Group By department_id;

Select * From department_salaries Where
    total_salary > 100000;
```

- Why use this?

→ Instead of computing SUM(salary) multiple times, we store results in a temp table for faster queries.

Key difference : CTEs vs Temporary Tables -

Feature	CTE, (with)	Temporary Tables.
Scope	Exists only during query execution.	Exists for the session or transaction.
Usage	Used in recursive queries and breaking complex queries.	Stores intermediate data for reuse.
Performance	Ideal for small datasets	Better for large datasets needing multiple queries.
Persistance	Disappears after	Persists until session

drawadorigonising

Performance Optimization.

1) Indexing for faster Querying

→ ~~Indexes~~ An index is a database structure that improves search speed by allowing the database to quickly locate records without scanning the entire table. It works like a table of contents in a book.

Types of Indexes

1) Primary Index → Automatically created for [Primary key]

2) Single-column Index → Created on a single column of a table.

3) Composite Index → Created on multiple columns, useful for queries that filter or sort by multiple columns.

4) Unique Index → Ensures that all values in the indexed column(s) are unique.

5) Full-text Index → Used for efficient text-based searches in large text fields.

6) Hash Index → Used for equality comparisons, often in memory-optimized tables.

7) Clustered & Non-clustered Indexes → Mostly in databases like SQL Server.

8) B-tree Index → The most common type, efficient for range queries and sorting.

drawandcoding

PAGE NO.....
DATE / /

When to Use Indexes:

- Column frequently used in WHERE clauses.
- Column used in Join conditions
- Column used in ORDER BY or GROUP BY clauses.

Best Practices:

- Avoid over-indexing as indexes consume storage and slow down write operations (INSERT, UPDATE, DELETE).
- Regularly monitor and remove unused or redundant indexes.
- Use composite indexes for queries that filter or sort by multiple columns.

2. Query Profiling and Analysis with Explain:

→ The Explain command is a powerful tool for analyzing and optimizing SQL queries. It provides insights into how the database engine executes a query, including the execution plan, indexes used, and potential bottlenecks.

→ EXPLAIN is a SQL command that helps analyze query executions plans, showing:

- Whether indexes are used
- The number of rows scanned
- Possible optimizations.

PAGE NO.....
DATE / /

table_name = user

id	name	email	age
1	Alice	alice@example.com	25
2	Bob	bob@example.com	30
3	Charlie	charlie@example.com	35
4	David	david@example.com	40
5	Eve	eve@example.com	45
Serial Primary key	Varchar(100)	Varchar(100) Unique	Int

(1) Query without index.

```
Select * From users Where email = 'bob@example.com';
```

(2) Analyze Query with [Explain]

```
Explain Select * From users Where email = 'bob@example.com';
```

(3) Add an index.

Create an index on the email column to speed up the query.

```
Create Index idx_email ON users(email);
```

(4) Repeat (2) again.

PAGE NO.....
DATE / /

Combining indexing and Explain for Optimization:

1. **Identify Slow Queries**

→ Use logging or monitoring tools to find queries with high execution times.

2. **Analyze with Explain**

→ Run `Explain` on the slow queries to identify bottlenecks.

3. **Add or Modify Indexes:**

→ Based on the `Explain` output, create or adjust indexes to optimize query performance.

4) **Re-analyze**

→ Use Explain again to verify that the changes improved the query plan.

dhavidofficial